

Advanced Runtime & Decoupling in C#

Reflection & Dynamic Object Creation

Reflection lets you examine and interact with type metadata while the program runs.

Inspecting Types at Runtime

```
using System.Reflection;

class Program
{
    static void Main()
    {
        Type type = typeof(DateTime);

        Console.WriteLine($"Name: {type.Name}");
        Console.WriteLine($"Namespace: {type.Namespace}");

        foreach (var method in type.GetMethods())
        {
            Console.WriteLine(method.Name);
        }
    }
}
```

Creating Objects Dynamically

```
using System.Reflection;

class Program
{
    static void Main()
    {
        // Get the Type object representing List<string>
        Type t = typeof(List<string>);

        // Dynamically create an instance of that type at runtime
        object instance = Activator.CreateInstance(t);

        // Cast the untyped object back to List<string> to use it normally
        var list = (List<string>)instance;
        list.Add("Hello Reflection");

        Console.WriteLine(list[0]); // Outputs: Hello Reflection
    }
}
```

This reflection code allows a program to create objects on the fly at runtime, even if the programmer didn't know what type of object would be needed beforehand.

Attributes - Adding Metadata

Attributes attach declarative information to classes, methods, or properties. They are a powerful way to embed metadata directly into your code that can be inspected at runtime via Reflection.

```
using System.Reflection;

public class DeveloperAttribute : Attribute
{
    public string Name { get; }
    public int Version { get; set; }

    public DeveloperAttribute(string name)
    {
        Name = name;
    }
}
```

Inherit from Attribute

Every custom attribute must extend the base `Attribute` class.

Define Properties

Store metadata using public properties - positional and named.

Constructor

Accept required values through the constructor for positional arguments.

Applying the Attribute

```
[Developer("John", Version = 1)]
class Calculator { }
```

Reading at Runtime

```
class Program
{
    static void Main()
    {
        var attr = typeof(Calculator)
            .GetCustomAttribute<DeveloperAttribute>();

        Console.WriteLine(attr.Name); // John
        Console.WriteLine(attr.Version); // 1
    }
}
```

Delegates - Type-Safe Method References

A delegate is a type that references a method with a specific signature. It enables methods to be passed as parameters, stored in variables, and invoked dynamically.

```
public delegate int Calculate(int x, int y);

class Program
{
    static int Add(int a, int b) => a + b;
    static int Multiply(int a, int b) => a * b;

    static void Main()
    {
        Calculate calc = Add;
        Console.WriteLine(calc(3, 4)); // 7

        calc = Multiply;
        Console.WriteLine(calc(3, 4)); // 12
    }
}
```

Declare the Delegate

Define the signature: return type and parameter types.

Assign a Method

Point the delegate to any method matching the signature.

Invoke Dynamically

Call the delegate like a function - it executes the target method.

Action & Func – Built-in Delegates

No need to declare custom delegates every time. .NET provides `Action` and `Func` as generic, ready-to-use delegate types.

Action – Returns void

Accepts up to 16 parameters, returns nothing.

```
class Program
{
    static void Main(string[] args)
    {
        Action<string> greet = GreetMethod;

        greet("Students");
    }

    static void GreetMethod(string name)
    {
        Console.WriteLine($"Hello, {name}");
    }
}
```

Func – Returns a Value

Last type parameter is the return type.

```
class Program
{
    static void Main(string[] args)
    {
        Func<int, int, int> subtract = SubtractMethod;

        int result = subtract(10, 3);
        Console.WriteLine(result);
    }

    static int SubtractMethod(int a, int b)
    {
        return a - b;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Func<int, double> calculateArea = CalculateCircleArea;

        double area = calculateArea(5);
        Console.WriteLine($"The area is: {area}");
    }

    static double CalculateCircleArea(int radius)
    {
        return Math.PI * radius * radius;
    }
}
```

Predicate – Boolean Checks

A `Predicate<T>` is a built-in delegate that takes one parameter and returns a `bool`. It is commonly used with collection methods like `FindAll`, `Find`, and `Exists`.

```
class Program
{
    static void Main(string[] args)
    {
        List<int> numbers = new() { 1, 2, 3, 4, 5, 6 };

        Predicate<int> isEven = IsEvenMethod;

        var evenNumbers = numbers.FindAll(isEven);

        foreach (var n in evenNumbers)
        {
            Console.Write(n + " ");
        }
    }

    static bool IsEvenMethod(int n)
    {
        return n % 2 == 0;
    }
}
```



Filter Collections

Use `FindAll` with a predicate to extract matching items.



Boolean Return

Predicate always returns `true` or `false` - perfect for conditions.



Lambda Syntax

Write concise inline predicates using lambda expressions.

Events – Loose Coupling with Delegates

Events wrap delegates to prevent direct invocation from outside the class. They provide a clean publish-subscribe mechanism where the publisher controls when notifications are sent.

```
public class TemperatureSensor
{
    public event Action<double> TemperatureChanged;

    private double _temperature;

    public double Temperature
    {
        get => _temperature;
        set
        {
            _temperature = value;
            TemperatureChanged?.Invoke(value);
        }
    }
}
```

1

Declare with `event`

Restricts external code from raising the event directly.

2

Raise with `?.Invoke()`

Safely notifies all subscribers only if any are attached.

3

Encapsulate the Trigger

The class controls *when* the event fires via property setters or methods.

Subscribing to Events

Subscriber Code

```
class Program
{
    static void Main()
    {
        var sensor = new TemperatureSensor();

        sensor.TemperatureChanged += AlertTemperatureHandler;

        sensor.Temperature = 25.5;
        sensor.Temperature = 30.0;
    }

    static void AlertTemperatureHandler(double temp)
    {
        Console.WriteLine($"Alert: {temp}°C");
    }
}
```

Output

```
Alert: 25.5°C
Alert: 30.0°C
```

Key Concept

Use `+=` to subscribe a handler. The handler is called automatically whenever the event is raised — no polling required.

Publisher-Subscriber Pattern

Multiple subscribers can listen without the publisher knowing who they are. This is the essence of loose coupling — the publisher broadcasts, and subscribers react independently.

Publisher

```
public class NewsPublisher
{
    public event Action<string> NewsPublished;

    public void Publish(string news)
    {
        Console.WriteLine("Publishing: " + news);
        NewsPublished?.Invoke(news);
    }
}
```

Multiple Subscribers

```
class Program
{
    static void Main()
    {
        var publisher = new NewsPublisher();

        publisher.NewsPublished += msg => Console.WriteLine($"Email sent: {msg}");
        publisher.NewsPublished += msg => Console.WriteLine($"SMS sent: {msg}");

        publisher.Publish("C# Class Started!");
    }
}
```

 **Output:** Publishing: C# Class Started! → Email sent: C# Class Started! → SMS sent: C# Class Started!